nscc
Information Technology
Generalist

PROG1400 – Study Guide

Course: PROG1400 – Introduction to Object-Oriented Programming

Instructor: Davis Boudreau

Midterm Type: Theory

Exam Type: Closed Book

- LO1: Describe Applications Using Core Object-Oriented Programming Principles**

What This Exam Tests: Your ability to **explain and describe** object-oriented design using:

- Pac-Man (shared case study)
- Your own Pac-Man-like game project

> ⚠ This exam is about **design thinking**, not coding syntax.

How to Study for This Exam

You should be able to:

- Explain OOP concepts **in your own words**
- Apply those concepts to **Pac-Man**
- Apply those concepts to **your own game**
- Talk about **objects and responsibilities**, not code

If you can explain your design clearly **without writing code**, you are ready.

Core Concepts You Must Know

1. **Object-Oriented Thinking**

Be able to explain:

- What an **object** is
- Why programs are broken into objects
- How objects represent **real things or roles** in a game

Example (Pac-Man):

- Pac-Man, Ghost, Maze, Pellet, GameController are objects
- Each has a **clear responsibility**

---

2. **Encapsulation**

You should be able to answer:

- What does encapsulation mean?
- Why is encapsulation important in a game?

Key idea to remember:

> An object manages its own data and behavior.

Pac-Man example:

- Pac-Man controls its position and movement
- Other objects should not directly change Pac-Man's position

Your game:

- Be ready to explain:
    - What data an object controls
    - Why other objects should not modify it directly

---

3. **Abstraction**

You should be able to explain:

- What abstraction means
- Why abstraction helps manage complexity

Key idea:

> Abstraction focuses on **what an object does**, not **how it does it**.

Pac-Man example:

- Pac-Man and Ghosts can both "move"
- They move differently, but at a high level they share behavior

Your game:

- Identify two objects that share behavior
- Explain how abstraction describes them at a higher level

---

4. **Inheritance**

You should understand:

- What inheritance is conceptually
- Why inheritance is used

Key idea:

> Inheritance reduces duplicated behavior by sharing common features.

Pac-Man example:

- A base character class could contain movement logic
- Pac-Man and Ghosts could inherit from it

You do **not** need to write code — only explain the idea.

---

5. **Polymorphism**

You should be able to describe:

- What polymorphism means
- How the same action can behave differently

Key idea:

> Different objects respond differently to the same action.

Pac-Man example:

- All ghosts can “move”
- Each ghost may move differently

Your game:

- Describe one behavior that changes depending on the object

---

6. **Aggregation**

You should be able to explain:

- What aggregation means in simple terms
- How objects can be grouped or owned

Key idea:

> Aggregation represents a “has-a” relationship.

Pac-Man example:

- The Maze has many Pellets
- The Game has many Ghosts

Be clear about **ownership**, not behavior.

---

7. **Interfaces (Conceptual)**

You should understand:

- What an interface represents
- Why interfaces are useful

Key idea:

> An interface defines what an object can do, not how it does it.

Game example:

- A movement interface could define `move()`
- Different objects implement movement differently

Again: **describe, don't code**.

How Pac-Man Fits the Exam

You must be able to:

- Explain **why Pac-Man fits OOP**
- Identify Pac-Man objects and responsibilities
- Use Pac-Man to explain OOP concepts clearly

If you cannot explain OOP using Pac-Man, review Week 2–4 materials.

How Your Game Fits the Exam

You must be able to:

- Clearly describe your game
- Name objects in your game
- Explain what each object is responsible for
- Compare your game design to Pac-Man

Your **Project Brief**, **Object Responsibilities Worksheet**, and **UML v1** are your best study tools.

Common Mistakes to Avoid

❌ Writing code ❌ Listing definitions without examples ❌ Confusing objects with screens or controls ❌ Giving one object too many responsibilities ❌ Forgetting to reference Pac-Man or your game

---

Self-Check Before the Exam

Ask yourself:

- Can I explain encapsulation without using the word "private"?

- Can I describe abstraction without writing code?
- Can I name at least 5 objects in my game and explain what they do?
- Can I compare my game to Pac-Man using OOP language?

If yes — you're ready.

Final Exam Tip

> This exam rewards **clear thinking**, **correct terminology**, and **strong examples**.

Think like a designer. Explain like a professional. Write clearly.

Support:

- Please do not hesitate to **reach out to your instructor before the exam if you have any concerns.**

Rubric – LO1 Exam

Total: 50 points

Criteria	Exceeds Expectations	Meets Expectations	Approaching Expectations	Below Expectations	Points
Understanding of OOP Principles	Provides clear, accurate explanations of encapsulation, abstraction, inheritance, polymorphism, aggregation, and interfaces using correct terminology	Mostly accurate explanations with only minor gaps	Partial understanding; explanations are vague or incomplete	Major misunderstandings or incorrect terminology	**/20**
Application to Pac-Man Case Study	Uses Pac-Man accurately and effectively to explain object responsibilities and OOP concepts	Uses Pac-Man with mostly correct examples; minor inaccuracies	References Pac-Man but explanations lack clarity	Little or incorrect reference to Pac-Man	**/15**

Criteria	Exceeds Expectations	Meets Expectations	Approaching Expectations	Below Expectations	Points
Application to Student Game Project	Clearly applies OOP concepts to the student's own game with well-defined objects and responsibilities	Applies OOP concepts with minor issues	Superficial or unclear application to student's game	Minimal or incorrect application to student's game	**/15**

Total Score: /50